

MarketTrend

Alexander Gutting, Jaime Espinola, Tsvetelina Hristova, Sinduja Manchi
CS 5100 Foundations Artificial Intelligence - Summer 2025
August 12th, 2025

Introduction:

Problem statement:

Forecasting stock market prices presents significant challenges due to the market's highly volatile and non-linear nature. Investors and analysts encounter many difficulties in forecasting future stock prices, since they are influenced by a wide range of factors including overall market conditions, company-specific events, and macroeconomic indicators. Traditional statistical methods are not always reliable in capturing the complex patterns, leading to inaccurate predictions. Consequently, there is a growing need for reliable AI and machine learning systems that can learn from historical stock market data and generate more reliable predictions of future stock price movements. For this reason, our team chose to focus on developing MarketTrend, an AI-driven system aimed at improving the accuracy of stock market predictions.

Motivation and goals:

Alex has a stock portfolio where he invests in multiple shares for various stocks such as Apple, Microsoft, and Uber. He also tends to keep up with the latest news and trends in the stock market on a daily basis. His interest in how the stock market works and changes over time influenced our decision to go with the idea of creating a stock market prediction model for a few specific S&P 500 stocks: AAL (American Airlines Group), AAPL (Apple), AAP (Advance Auto Parts), ABBV (Abbvie), and ABC (American Broadcasting Company)). Our main goal for this project was to create a proper linear regression model using various charts generated from the matplotlib and seaborn modules. Since we used existing data from thousands of S&P 500 stock performances on specific dates, we decided to find a shorter, more optimal solution for the stock data. We want to visualize the key points of the data; however, since there is so much data, we output a chart listing the overall data of all of the stock performances with the highest and lowest recorded lows in a span of 5 years (2013 to 2018). We did this so we could get a good idea of what we wanted our initial dataset to look like. We also wanted to calculate and visualize the metrics that are appropriate for a normal regression model. Those metrics are the Mean Squared Error, R-Squared Value, and Absolute Error. We used the following metrics to find a fairly accurate prediction for all of the recorded stock performances based on real-time data.

Background:

Related work:

Sample Linear Regression Model: emineyetm. (2022, December 4). *Multiple Linear Regression in Python*. Kaggle.com; Kaggle.

<https://www.kaggle.com/code/emineyetm/multiple-linear-regression-in-python/notebook>

Sample Logistic Regression Model: ellezee. (2025, July 14). *Sentiment Analysis Using Logistic Regression*. Kaggle.com; Kaggle.

<https://www.kaggle.com/code/ellezee/sentiment-analysis-using-logistic-regression>

Brief explanation of AI/ML concepts used:

We decided to use the following concepts:

1. Preprocess the data and handling missing values, remove any duplicates, and encode categorical variables to prepare the data.
2. Use linear regression.
3. Implement the following evaluation metrics; Mean Squared Error (MSE), Mean Absolute Error (MAE) and R2 to assess the model performance.
4. Visualized data trends and differences, such as actual vs. predicted plot, residual plot, and distribution graph.

Methodology:

Tools and frameworks:

Our group used Google CoLab to write code and create various drafts and prototypes of our desired model. Then, we used Python/Jupyter Notebook for our finished model. In addition, we used a variety of modules to achieve our goals for this project. Those modules include pandas/numpy (the basics), kagglehub/os (for importing the data from the Kaggle website), sklearn (for using Linear Regression and calculating the appropriate metrics), and matplotlib/seaborn (for visualizing the data). Lastly, we used a .csv file called "all_stocks_5yr.csv" from Kaggle to import, modify, and output the stock data in our dataset.

Datasources and preprocessing:

We used the S&P 500 Stock Data dataset from Kaggle:

Link: <https://www.kaggle.com/datasets/camnugent/sandp500>

Dataset Details:

Filename: all_stocks_5yr.csv

Columns:

Date

Open: Price in USD of the stock at market open

High: Highest price of the stock on the given date

Low: Lowest price of the stock on the given date

Close: Close price of the stock on the given date

Volume: Number of shares that were traded of the stock on the given

date

Name: Ticker identifier of the stock

Size: 5 years of daily stock prices for all S&P companies - 7 Columns, 619040 rows

Preprocessing Steps:

Loaded the dataset and inspected structure (df.info(), df.head()).
Removed missing values in open, high, and low columns using df.dropna().
Removed duplicate records using df.drop_duplicates().
Encoded categorical stock Name using LabelEncoder().
Selected features: open, high, low, volume, Name_numeric.
Set close as the target variable.
Split data into 80% training and 20% testing sets.

Algorithms/models/codes used (diagrams included):

To evaluate the model, we used MSE, r^2 , and MAE, detailed below. Mean squared error (MSE) score is the average of the squared differences between the actual and predicted results. A lower value indicates a better fit because it shows the actual values are closer to the predicted values. The value of 0.4697 indicates a good fit. The r^2 score measures how well the model explains the variability of the target variable. A value of 1 would indicate that the model predicts the data perfectly. Our value is very close to 1, showing a good fit. The mean absolute error (MAE) measures the absolute difference between the actual values and the predicted values from the model. In general, a smaller MAE is better. The MAE value indicates by how many units off are the predictions. In our case this would mean we are off by 0.32 cents, which is pretty good.

Results:

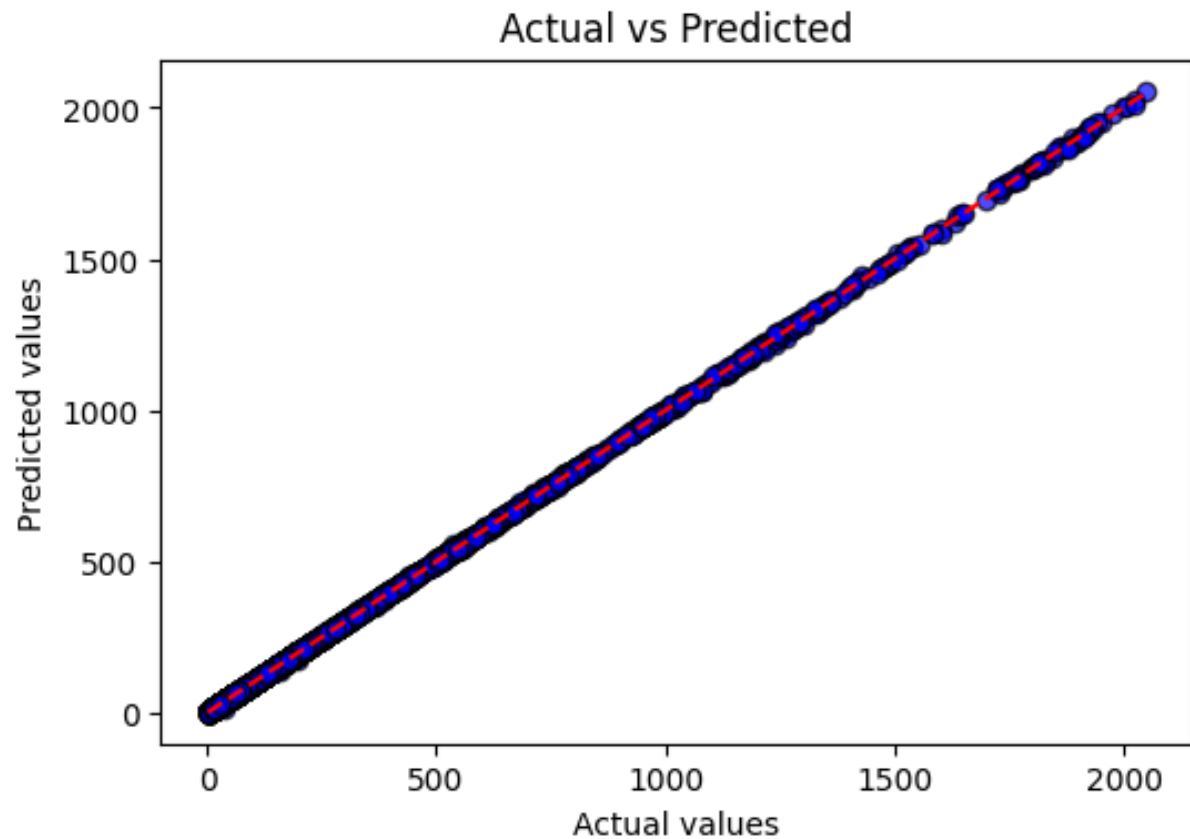
Model evaluation metrics:

Mean Squared Error (MSE): 0.469
 R^2 (Coefficient of Determination): 1.00
Mean Absolute Error (MAE): 0.34

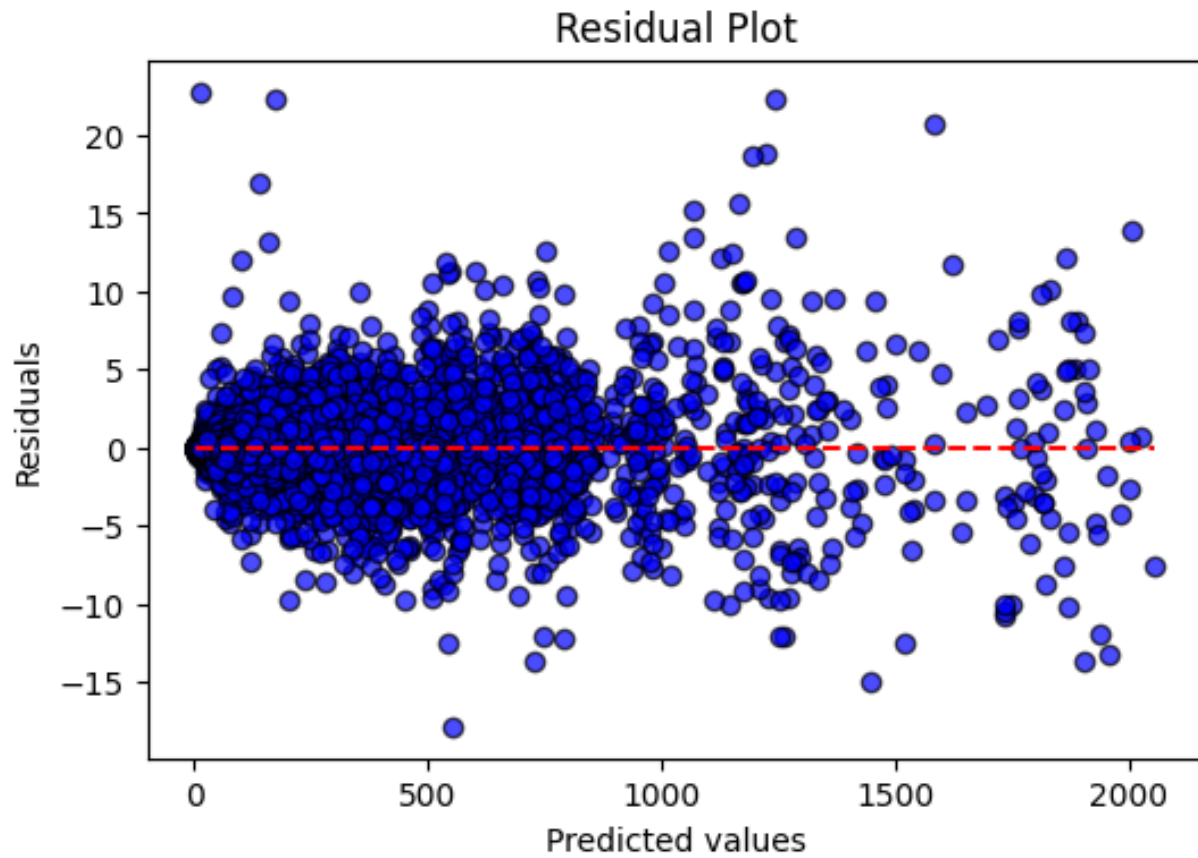
These metrics indicate a significantly accurate regression model.

Vizualizations:

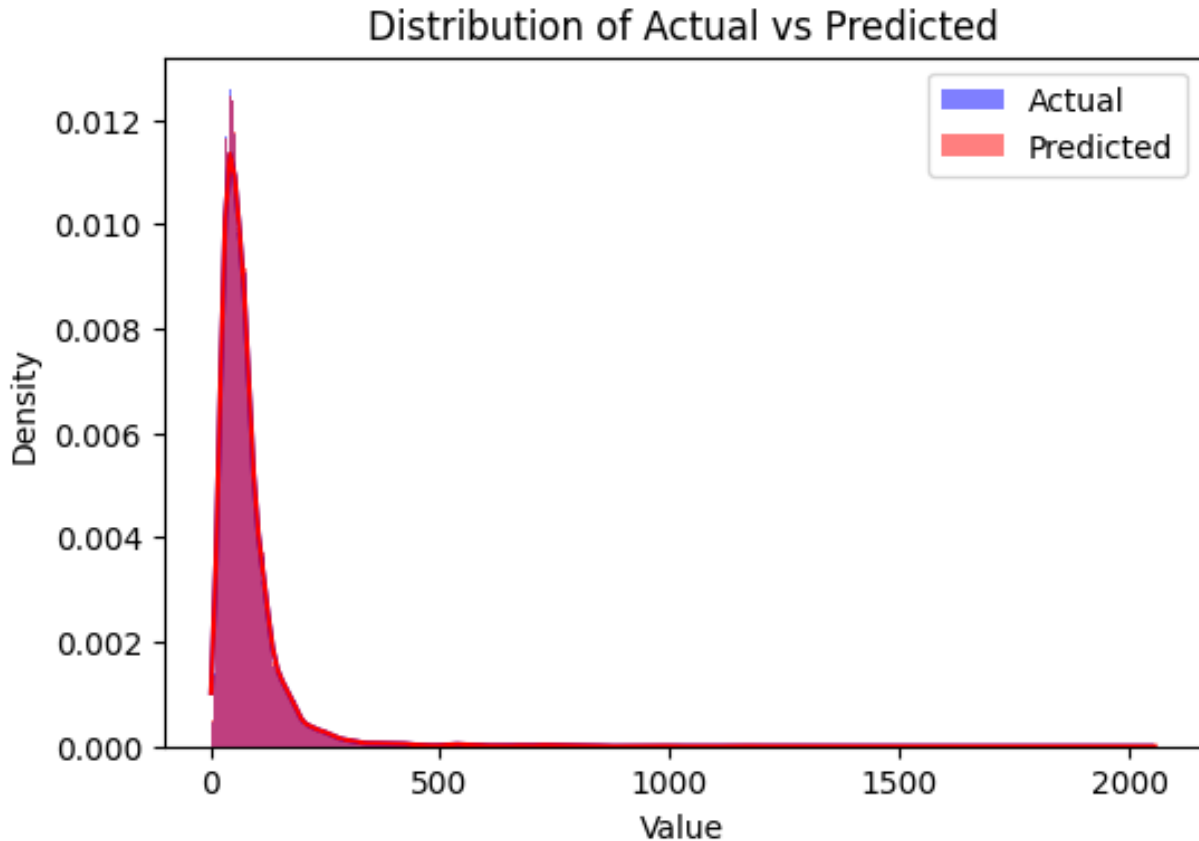
Actual vs Predicted Scatter Plot shows the correlation between the true and predicted prices:



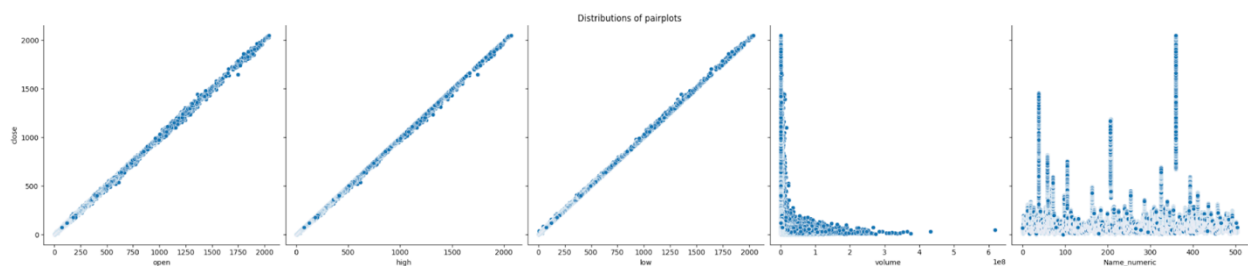
Residual Plot displays the error distribution. In a residual plot, it is best to see a random but evenly scattered pattern of points around the horizontal axis - this shows that the errors are random and that there is no major bias. As seen below, the plot looks good.



Prediction Distribution Plot compares the density between actual and predicted prices. In a good fit, the two plots overlay each other almost completely to indicate a close match. As seen below, the two plots are very closely matched and therefore the model is a good fit.

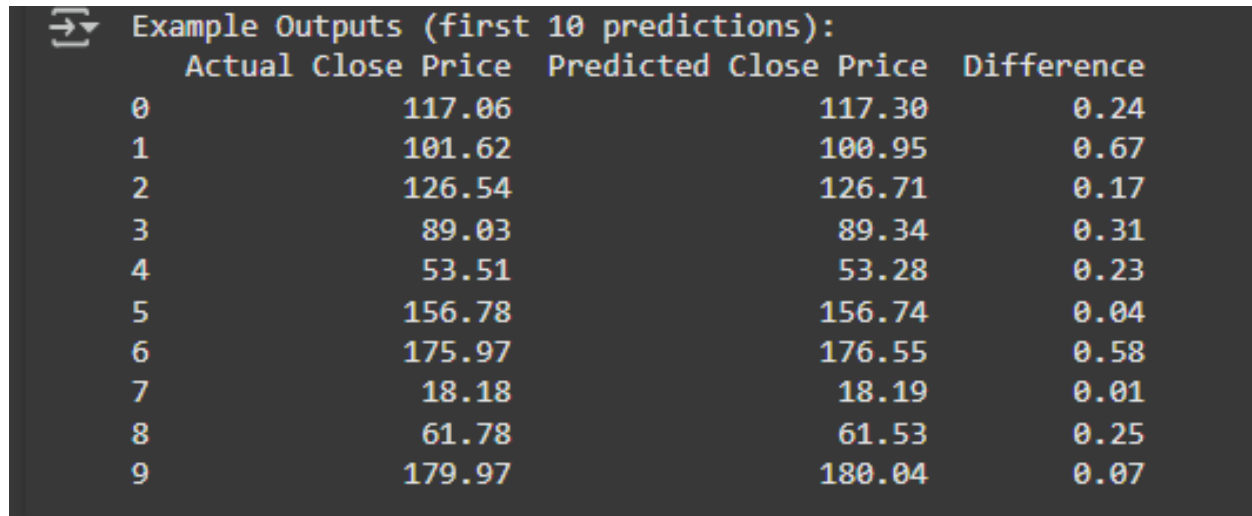


5 x 1 pairplot that compares the feature and target data from the dataframe. In each of the five plots below, the relationship between a feature (such as open price, high price, low price, volume, or name) and the close price is shown. Based on the plots below, the open, high, and low prices are good predictors of the close price. Higher open, high, and low prices indicate a higher close price. The correlation between the volume and close price indicates that most stocks have a low volume and a low price. This skews the data and makes it difficult to establish a strong correlation between volume and close price. Finally, the name graph shows that there is no correlation between name and closing price. This makes sense because the closing price is not related to the name of the stock.



Example outputs:

Finally, below is a dataframe that summarizes the actual closing price versus the predicted price for various stocks. It is a nice visual to show how well the model is working to predict the prices. As seen below, most of the price differences are off by less than 50 cents.

A screenshot of a Jupyter Notebook interface. On the left, there is a file explorer icon and a dropdown menu showing 'Example Outputs (first 10 predictions):'. The main area displays a table with four columns: 'Actual Close Price', 'Predicted Close Price', and 'Difference'. The table contains 10 rows of data, indexed from 0 to 9. The 'Actual Close Price' and 'Predicted Close Price' columns are right-aligned, while the 'Difference' column is left-aligned. The values in the 'Difference' column are all small, ranging from 0.01 to 0.67.

	Actual Close Price	Predicted Close Price	Difference
0	117.06	117.30	0.24
1	101.62	100.95	0.67
2	126.54	126.71	0.17
3	89.03	89.34	0.31
4	53.51	53.28	0.23
5	156.78	156.74	0.04
6	175.97	176.55	0.58
7	18.18	18.19	0.01
8	61.78	61.53	0.25
9	179.97	180.04	0.07

Discussion:

Limitations:

Since there were so many stocks in the data we analyzed, we decided to only output the data from the first ten stocks in our updated dataframe in a chart at the end of our model.

We believe that a linear regression model might not capture the full complexity of the stock market, since it assumes a linear relationship. Some other limitations could be overfitting potentially and external factors such as macroeconomics.

Interpretation of results:

We output our final results in several different formats visually, both in charts and graphs. We used the seaplot (sns) module to compare the information in our dataframe and used the matplotlib (plt) module to output those comparisons as either basic charts or plot distributions of our data. We compared some test data with just 10 predictions using the standard 80% training data vs 20% testing data technique to get a solid idea of how future data was predicted. We also calculated the residuals and the distribution of the predictions themselves. Then, we created a pairplot graph that compares each variable (category) of the dataframe. We wanted a variety of comparisons between certain categories as well as metrics for diversity to ensure that our project works the way we wanted it to.

Potential improvements:

One way we could've made the pre-production process more efficient is agreeing on what specific parts or pieces of code of the project we were supposed to work on before starting the actual project. However, despite our code as a whole working the way we initially wanted it to, the production process was more scattered and unorganized because we weren't initially sure which group member was going to do what.

Conclusion:

Summary of achievements:

We were successfully able to get the necessary metric values for our S&P 500 stock performances that we were looking for. We also collected and compared various data in various charts and graphs after the analysis process. Overall, we were very satisfied with our results.

Lessons learned:

Linear regression is great for a sturdy foundation; however, it does not support

complexity, which is needed for longer-term prediction.

The most important part of the project was making sure we completed the data preprocessing prior to building an accurate model.

References:

- Kaggle. (2018). S&P 500 Stock Data. Retrieved from <https://www.kaggle.com/datasets/camnugent/sandp500>
- mervinpraison. (2019, August 13). *Pairplots in Python*. Kaggle.com; Kaggle. <https://www.kaggle.com/code/mervinpraison/pairplots-in-python>